

SQL

Databases, CRUD, Keys and Joins: Complete Lecture with Practice Sets

Ananda Rimal

SQL Lecture

August 10, 2026

What We Will Learn

1. Why companies need SQL, database, DBMS and tables
2. **Part 1: CRUD in full** Create, Insert, Read in depth (WHERE, AND, OR, IN, BETWEEN, LIKE, ORDER BY), Update, Delete
3. **Practice Set 1:** 20 questions on a new shop database
4. **Part 2: Keys** Primary Key and Foreign Key, why we need them, orphans, cascade
5. **Practice Set 2:** build a hospital database with keys
6. **Part 3: Joins** Inner, Left, Right, Full, Self, Exclusive
7. **Practice Set 3:** joins on a library database
8. **Part 4: Aggregates** COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING
9. **Practice Set 4:** report questions

Rule of this class: first we learn the full topic, then we practice with many questions.

Why Do Companies Need SQL?

Think about a bank in Kathmandu

- ▶ Thousands of customers deposit and withdraw every day.
- ▶ Every transaction must be stored **safely**, found **quickly**, never lost or duplicated.
- ▶ Excel breaks down: lakhs of rows, many people editing at once, no rules to stop wrong data.

More examples:

- ▶ **Daraz**: products, orders, payments, delivery status
- ▶ **Hospital**: patients, doctors, appointments, reports
- ▶ **College**: students, courses, marks, fees

The answer

Companies store data in **databases**, and **SQL** is the language to save, search, change and summarize that data.

Database and DBMS

- ▶ A **database** is a collection of data in a format that can be easily accessed (digital).
- ▶ A software application used to manage our DB is called **DBMS** (Database Management System).



Types of databases

- ▶ **Relational**: data stored in tables. MySQL, PostgreSQL, Oracle, SQL Server. We use **SQL** with these.
- ▶ **Non-relational (NoSQL)**: data not stored in tables. MongoDB.

What is SQL?

Structured Query Language

SQL is a programming language used to interact with relational databases. It performs CRUD operations:

Letter	Operation	Command	Meaning
C	Create	INSERT	put new data in
R	Read	SELECT	take data out
U	Update	UPDATE	change existing data
D	Delete	DELETE	remove data

Types of SQL commands

DDL: create, alter, rename, truncate, drop DQL: select
DML: insert, update, delete DCL: grant, revoke TCL: commit, rollback

Part 1

CRUD: Create, Read, Update, Delete

First we learn all of CRUD. The 20-question practice set comes after.

Creating Our First Database and Table

```
CREATE DATABASE college;
USE college;

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    grade CHAR(2)
);

SHOW DATABASES;    SHOW TABLES;
```

More database queries:

- ▶ `CREATE DATABASE IF NOT EXISTS college;` (safe create)
- ▶ `DROP DATABASE IF EXISTS college;` (safe delete)
- ▶ Datatypes: `INT`, `VARCHAR(50)`, `CHAR(2)`, `DECIMAL(10,2)`, `DATE`, `BOOLEAN`

C of CRUD: INSERT

```
INSERT INTO students VALUES
(1, 'Ram Sharma', 20, 'Kathmandu', 'A'),
(2, 'Sita Rai', 22, 'Pokhara', 'B'),
(3, 'Hari Thapa', 19, 'Kathmandu', 'A'),
(4, 'Gita Magar', 21, 'Chitwan', 'C'),
(5, 'Krishna Gurung', 23, 'Pokhara', 'B');

SELECT * FROM students;
```

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
2	Sita Rai	22	Pokhara	B
3	Hari Thapa	19	Kathmandu	A
4	Gita Magar	21	Chitwan	C
5	Krishna Gurung	23	Pokhara	B

This table is our example for the whole lecture.

INSERT: More Examples

```
-- insert only some columns (grade becomes NULL)
INSERT INTO students (student_id, name, age, city)
VALUES (6, 'Anita KC', 20, 'Bhaktapur');

-- insert multiple rows at once
INSERT INTO students VALUES
(7, 'Deepak Oli', 22, 'Kathmandu', 'B'),
(8, 'Mina Pandey', 19, 'Pokhara', 'C');
```

ID	Name	Age	City	Grade
6	Anita KC	20	Bhaktapur	NULL
7	Deepak Oli	22	Kathmandu	B
8	Mina Pandey	19	Pokhara	C

Columns you skip are filled with **NULL** (means: no value, empty, unknown).

R of CRUD: SELECT

Select and view **ALL** columns, or only some

```
SELECT * FROM students;  
SELECT name, city FROM students;
```

Name	City
Ram Sharma	Kathmandu
Sita Rai	Pokhara
Hari Thapa	Kathmandu
Gita Magar	Chitwan
Krishna Gurung	Pokhara

- ▶ * means every column. Naming columns returns only those.
- ▶ SELECT only reads. It never changes the table.

WHERE Clause: Filtering Rows

WHERE means: **only give me the rows where this condition is true.**
The database checks the condition on every row, one by one.

```
SELECT * FROM students WHERE city = 'Kathmandu';  
SELECT * FROM students WHERE age > 20;
```

Result of the first query:

ID	Name	Age	City	Grade
1	Ram Sharma	20	Kathmandu	A
3	Hari Thapa	19	Kathmandu	A

Operators usable in WHERE

Comparison: = != > >= < <= Logical: AND, OR, NOT, IN, BETWEEN, LIKE

Arithmetic: + - * / %

Operators: AND and OR

AND: both conditions must be true

```
SELECT name, age, grade FROM students  
WHERE age > 20 AND grade = 'B';
```

Sita (22, B) and Krishna (23, B) pass. Ram fails: age 20 is not > 20.

OR: one of the conditions is enough

```
SELECT name, city FROM students  
WHERE city = 'Chitwan' OR grade = 'A';
```

Name	City
Ram Sharma	Kathmandu
Hari Thapa	Kathmandu
Gita Magar	Chitwan

Memory trick: AND shrinks the result, OR grows the result.

Operator IN: Value From a List

Why do we need IN? Writing many ORs is long and ugly:

```
-- the long way
SELECT name FROM students
WHERE city = 'Kathmandu' OR city = 'Pokhara';

-- the short way: IN
SELECT name FROM students
WHERE city IN ('Kathmandu', 'Pokhara');

-- opposite: everything NOT in the list
SELECT name FROM students
WHERE city NOT IN ('Kathmandu', 'Pokhara');
```

- ▶ IN returns rows where the value matches **any** item in the list.
- ▶ First two queries: Ram, Sita, Hari, Krishna. NOT IN: only Gita (Chitwan).

Operator BETWEEN: A Range

Why do we need BETWEEN? For ranges, two comparisons become one clean line:

```
-- the long way
SELECT name, age FROM students
WHERE age >= 20 AND age <= 22;

-- the short way: BETWEEN
SELECT name, age FROM students
WHERE age BETWEEN 20 AND 22;
```

Name	Age
Ram Sharma	20
Sita Rai	22
Gita Magar	21

- ▶ BETWEEN is **inclusive**: both ends 20 and 22 are counted.
- ▶ Works on numbers and dates: WHERE dob BETWEEN '2000-01-01' AND '2005-12-31'

Operator LIKE: Pattern Matching

Why do we need LIKE? = needs the exact value. LIKE searches by **pattern**, like search boxes do.

```
SELECT name FROM students WHERE name LIKE 'R%';
SELECT name FROM students WHERE name LIKE '%i';
SELECT name FROM students WHERE name LIKE '%ar%';
SELECT name FROM students WHERE name LIKE '_ita%';
```

Pattern	Meaning	Matches here
'R%'	starts with R	Ram Sharma
'%i'	ends with i	Sita Rai
'%ar%'	contains ar	Ram Sharma, Hari Thapa, Gita Magar
'_ita%'	any 1 char, then ita	Sita Rai, Gita Magar

- ▶ % = any number of characters (even zero). _ = exactly one character.

ORDER BY, DISTINCT, LIMIT

```
SELECT name, age FROM students ORDER BY age DESC;  
SELECT DISTINCT city FROM students;  
SELECT name, age FROM students ORDER BY age DESC LIMIT 3;
```

ORDER BY age DESC

Name	Age
Krishna Gurung	23
Sita Rai	22
Gita Magar	21
Ram Sharma	20
Hari Thapa	19

DISTINCT city

City
Kathmandu
Pokhara
Chitwan

DESC = big to small, ASC = small to big (default). LIMIT keeps only the first N rows.

U of CRUD: UPDATE

```
UPDATE students
SET grade = 'A'
WHERE student_id = 4;

-- update many rows at once
UPDATE students
SET grade = 'B'
WHERE city = 'Pokhara';
```

Before

ID	Name	Grade
4	Gita Magar	C

After

ID	Name	Grade
4	Gita Magar	A

Danger!

UPDATE students SET grade = 'A'; without WHERE changes **every** row. Write the WHERE first, then the SET.

D of CRUD: DELETE (and its Cousins)

```
DELETE FROM students WHERE student_id = 8;  
DELETE FROM students WHERE grade = 'C';
```

Command	Removes	Table stays?
DELETE ... WHERE	matching rows	yes
DELETE FROM t;	all rows	yes
TRUNCATE TABLE t;	all rows (faster)	yes
DROP TABLE t;	rows + structure	no

ALTER: changing the schema

```
ALTER TABLE students ADD COLUMN marks INT;  
ALTER TABLE students DROP COLUMN marks;  
ALTER TABLE students RENAME TO stu;
```

Danger!

DELETE without WHERE deletes everything. Same rule as UPDATE.

Practice Set 1: Your New Database

CRUD is finished. Now **you** build everything from zero. New dataset for all 20 questions:

```
CREATE DATABASE shop;  
USE shop;  
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    pname VARCHAR(50),  
    price INT,  
    category VARCHAR(30),  
    stock INT  
);
```

Data you will insert (Q2):

id	pname	price	category	stock
1	Rice Bag	2200	Grocery	50
2	Laptop	85000	Electronics	10
3	Mobile	30000	Electronics	25
4	Noodles	25	Grocery	500
5	Shampoo	350	Beauty	80
6	TV	55000	Electronics	8

Practice Set 1: Create and Insert (Q1–Q5)

C: Create and Insert

1. Create the shop database and the products table exactly as shown. Confirm with `SHOW TABLES;`.
2. Insert all 6 products in **one single** `INSERT` statement.
3. Insert product 7, giving only `product_id` and `pname` ('Pen'). Which columns become `NULL`?
4. Try to insert another product with `product_id` = 3. Write down the exact error and explain why it happened.
5. Insert two more products of your choice in one statement: one Grocery item and one Beauty item.

Practice Set 1: Read (Q6–Q15)

R: Read (one query each)

6. Show all columns of all products.
7. Show only pname and price.
8. Show products with price above 1000.
9. Show Electronics products with stock less than 20 (AND).
10. Show products that are Grocery **or** cost less than 500 (OR).
11. Show products in categories Grocery and Beauty, using IN.
12. Show products with price between 300 and 40000. Is 350 included? Why?
13. Show products whose name starts with M.
14. Show products whose name contains the letter o.
15. Show the 2 most expensive products (which two keywords together?).

Practice Set 1: Update and Delete (Q16–Q20)

U and D: Update and Delete

16. The Laptop price dropped to 79000. Update it using its id.
17. Festival offer: reduce the price of **every** Electronics product by 10%.
Hint: `SET price = price * 0.9`.
18. Ten Rice Bags were sold. Decrease its stock by 10 with one query.
19. Delete the product named Pen. Then delete all products with stock below 10. How many rows did each query remove?
20. Run `SELECT` to check the final table. Then answer: which command would empty the table but keep it, and which would remove the table completely?

Check yourself: run a `SELECT` before and after every `UPDATE` and `DELETE`.

Part 2

Keys: Primary Key and Foreign Key

First the full theory, then Practice Set 2 where you build a hospital database.

Why Do We Need Keys? The Two Ram Problem

Imagine our college has **two** students named **Ram Sharma**.

Name	Age	City	Grade
Ram Sharma	20	Kathmandu	A
Ram Sharma	21	Pokhara	C

- ▶ The exam office says: "Update Ram Sharma's grade to B." **Which Ram?**
- ▶ `UPDATE students SET grade='B' WHERE name='Ram Sharma';` changes **both**. One student gets a wrong grade!
- ▶ Names, cities, ages can repeat. We need one column that can **never** repeat.

The fix

Give every row a unique identity number. That column is the **Primary Key**. That is why your college gives you a roll number instead of calling you only by name.

Keys: Primary Key

Definition

A **Primary Key (PK)** is a column (or set of columns) that **uniquely identifies** each row. There is only 1 PK per table and it should be NOT NULL.

Properties:

- ▶ **UNIQUE**: no two rows share a value
- ▶ **NOT NULL**: can never be empty
- ▶ **One per table**

student_id (PK)	name	age
101	Alice	20
102	Bob	22
103	Charlie	21

More examples of good PKs:

- ▶ citizenship number, roll number
- ▶ account number in a bank
- ▶ order id in Daraz, SIM number

Can name be a PK? No! Two students can share a name, so it is not unique.

Primary Key in Action: The DB Protects You

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
INSERT INTO students VALUES (101, 'Alice'); -- OK  
INSERT INTO students VALUES (101, 'Bob');  
-- ERROR: Duplicate entry '101' for key 'PRIMARY'  
  
INSERT INTO students VALUES (NULL, 'Charlie');  
-- ERROR: Column 'student_id' cannot be null
```

- ▶ The PK is a **promise**: every row can always be found by exactly one value.
- ▶ UPDATE and DELETE become safe: WHERE student_id = 101 touches exactly one row, never the wrong Ram.

Constraints: Rules for Data

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    age INT CHECK (age >= 16),  
    city VARCHAR(50) DEFAULT 'Kathmandu'  
);
```

Constraint	Rule
NOT NULL	column cannot have a null value
UNIQUE	all values in column are different
PRIMARY KEY	unique + not null, only one
FOREIGN KEY	value must exist in parent table
CHECK	limits the values allowed
DEFAULT	sets the default value of a column

Why Do We Need a Second Table (and FK)?

Try storing courses **inside** the students table:

id	name	city	course
101	Alice	Kathmandu	Math
101	Alice	Kathmandu	Physics
102	Bob	Pokhara	Math

Problems with one big table:

- ▶ Alice's name and city are **repeated** in every course row (wasted space).
- ▶ Alice moves to Lalitpur: we must update **many** rows. Miss one and the data disagrees with itself.
- ▶ Delete Alice's last course and Alice herself disappears from the database!

The fix

Split into two tables: `students` (each person once) and `enrollments` (who takes what). Then **link** them. That link is the **Foreign Key**.

Keys: Foreign Key

Definition

A **Foreign Key (FK)** is a column in a table that **refers to the Primary Key of another table**. There can be multiple FKs. FKs can have duplicate and NULL values.

Parent: students

student_id (PK)	name
101	Alice
102	Bob
103	Charlie

Child: enrollments

enroll_id (PK)	student_id (FK)	course
1	101	Math
2	101	Physics
3	102	Math

- ▶ 101 repeats: Alice takes two courses. That is allowed for FKs.
- ▶ PK lives in the **parent** table, FK lives in the **child** table.

Database Design: The Orphan Problem

Without a foreign key, the child table accepts anything:

```
-- students has only ids 101, 102, 103
INSERT INTO enrollments VALUES (5, 999, 'Chemistry');
-- ACCEPTED! But who is student 999? Nobody!
```

enroll_id	student_id	course
1	101	Math
2	101	Physics
3	102	Math
5	999 ???	Chemistry

Row 5 is an **orphan record**: a child row whose parent does not exist. Orphans cause wrong reports, application crashes, and data nobody can trust.

The FK Fixes It + Cascade

```
CREATE TABLE enrollments (  
  enroll_id INT PRIMARY KEY,  
  student_id INT,  
  course VARCHAR(50),  
  FOREIGN KEY (student_id) REFERENCES students(student_id  
    )  
  ON DELETE CASCADE  
  ON UPDATE CASCADE  
);  
  
INSERT INTO enrollments VALUES (5, 999, 'Chemistry');  
-- ERROR: foreign key constraint fails. REJECTED!
```

FK option	Effect on child rows
RESTRICT (default)	block the action if children exist
ON DELETE CASCADE	delete children automatically
ON UPDATE CASCADE	update children's FK automatically
ON DELETE SET NULL	keep children, FK becomes NULL

Banks prefer RESTRICT: never delete, just mark records inactive.

Practice Set 2: Your New Database with Keys

Keys are finished. New dataset: a **hospital**.

```
CREATE DATABASE hospital;
USE hospital;

CREATE TABLE doctors (
    doctor_id INT PRIMARY KEY,
    dname VARCHAR(50) NOT NULL,
    department VARCHAR(30)
);

CREATE TABLE appointments (
    appt_id INT PRIMARY KEY,
    doctor_id INT,
    patient_name VARCHAR(50),
    fee INT,
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
    ON DELETE CASCADE
);
```

Parent = doctors, child = appointments. Keep this open for all questions.

Practice Set 2: Primary Key (Q1–Q5)

Primary Key questions

1. Create the `hospital` database and both tables exactly as shown.
2. Insert 3 doctors: (1, Dr. Sharma, Heart), (2, Dr. Rai, Bone), (3, Dr. KC, Eye).
3. Try to insert a doctor with `doctor_id = 2` again. Write down the error.
4. Try to insert a doctor with `NULL` id, and one with `NULL` name. Which constraint blocks each?
5. For each real system, decide a good PK and say why name is not enough: patients in this hospital, products in Daraz, citizens of Nepal.

Practice Set 2: Foreign Key (Q6–Q12)

Foreign Key questions

6. Insert 4 appointments for existing doctors, two of them for doctor 1. Why is the repeated `doctor_id` allowed?
7. Try to insert an appointment with `doctor_id = 99`. What happens and why is this good?
8. Which table is parent and which is child? Where does the FK live?
9. Delete doctor 1. What happened to doctor 1's appointments, and which FK option caused that?
10. Recreate the FK with `RESTRICT` instead. Now try deleting a doctor who has appointments. What happens?
11. Real life: Daraz has customers and orders. Draw the parent, the child, and the FK column.
12. Why can an FK repeat and be `NULL` while a PK cannot? One sentence each.

Part 3

Joins

First all join types, then Practice Set 3 on a library database.

Why Do We Need Joins?

Good design split our data into two tables. But now the exam office asks:

"Give me a list of student NAMES with their COURSES."

- ▶ Names live in `students`. Courses live in `courses`.
- ▶ No single table has the full answer.
- ▶ A **JOIN** glues rows from two tables back together using the related column (the FK).

Join is used to combine rows from two or more tables, based on a related column

Design splits the data (to keep it clean), joins combine it again (to read it). Both are needed.

Joins in SQL: Setup

Our two tables for every join slide:

students	
student_id	name
1	Ram
2	Sita
3	Hari
4	Gita
5	Krishna

courses		
course_id	course	student_id
101	Math	1
102	Science	1
103	English	2
104	History	2
105	Math	3
106	Computer	5

- ▶ Related column: `student_id` in both tables.
- ▶ **Gita (id 4) has no course**, and every course has a student. Watch Gita on each slide!

Inner Join

Returns records that have matching values in both tables

```
SELECT s.name, c.course
FROM students AS s
INNER JOIN courses AS c
ON s.student_id = c.student_id
;
```

How to read it:

- ▶ take each student row
- ▶ find courses with the same id
- ▶ output one row per match
- ▶ **no match = no row**

Gita is excluded: no matching course, no row.

Result (6 rows)

name	course
Ram	Math
Ram	Science
Sita	English
Sita	History
Hari	Math
Krishna	Computer

Left Join

Returns ALL records from the left table, and the matched records from the right table

```
SELECT s.name, c.course
FROM students AS s
LEFT JOIN courses AS c
ON s.student_id = c.student_id
;
```

- ▶ left table = the one written after FROM (students)
- ▶ every student appears, match or not
- ▶ missing side is filled with NULL

Result (7 rows)

name	course
Ram	Math
Ram	Science
Sita	English
Sita	History
Hari	Math
Gita	NULL
Krishna	Computer

Gita is included with NULL: no course, but every student appears.

Right Join

Returns **ALL** records from the right table, and the matched records from the left table

```
SELECT c.course, s.name
FROM students AS s
RIGHT JOIN courses AS c
ON s.student_id = c.student_id
;
```

- ▶ right table = courses
- ▶ every course appears
- ▶ here every course has a student, so no NULLs appear

Result (6 rows)

course	name
Math	Ram
Science	Ram
English	Sita
History	Sita
Math	Hari
Computer	Krishna

Every course appears. **Gita is excluded again**: she is not attached to any course row.

Full Join (MySQL: LEFT UNION RIGHT)

Returns all records when there is a match in either left or right table

```
SELECT s.name, c.course
FROM students AS s
LEFT JOIN courses AS c
ON s.student_id = c.student_id
UNION
SELECT s.name, c.course
FROM students AS s
RIGHT JOIN courses AS c
ON s.student_id = c.student_id;
```

Result (7 rows)

name	course
Ram	Math
Ram	Science
Sita	English
Sita	History
Hari	Math
Gita	NULL
Krishna	Computer

Everything from both sides. MySQL has no FULL JOIN keyword, so we take LEFT UNION RIGHT.

Join Comparison Summary

Join	Rows here	Gita (no course)	Returns
INNER	6	excluded	only matches
LEFT	7	NULL values	all left + matches
RIGHT	6	excluded	all right + matches
FULL	7	NULL values	all from both

How to choose the right join

Ask: **whose rows must ALL appear?**

All students, even without courses: LEFT JOIN from students.

Only real matches: INNER JOIN.

Everything with gaps shown as NULL: FULL JOIN.

Practice Set 3: Your New Database for Joins

Joins are finished. New dataset: a **library**.

```
CREATE DATABASE library;
USE library;

CREATE TABLE members (
    member_id INT PRIMARY KEY,
    mname VARCHAR(50),
    city VARCHAR(30)
);

CREATE TABLE loans (
    loan_id INT PRIMARY KEY,
    member_id INT,
    book_title VARCHAR(80),
    fine INT,
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);
```

Insert 5 members (make sure **one member has no loans**) and 8 loans before starting.

Practice Set 3: Joins (Q1–Q5)

Join questions, first half

1. INNER JOIN members and loans. **Predict** the row count before running.
2. LEFT JOIN showing all members with their books. Who shows NULL and why?
3. Swap the two tables in the LEFT JOIN. Which earlier join does it now behave like?
4. RIGHT JOIN from members. What appears, and is any row NULL? Why not?
5. Write the FULL JOIN using UNION. How many rows?

Tip: before every join, ask whose rows must ALL appear.

Practice Set 3: Joins (Q6–Q10)

Join questions, second half

6. Exclusive join: members who borrowed **nothing**. Which IS NULL check?
7. Show member names with book titles only for fines above 50 (join + WHERE).
8. Delete one loan, run the INNER JOIN again. Which rows disappeared?
9. In your own words: when do you choose LEFT over INNER? Give a library example.
10. Bonus self join: add a `referred_by` column to members (who invited whom) and list inviter with invited.

Part 4

Aggregate Functions, GROUP BY, HAVING

First the full theory, then Practice Set 4 with report questions.

Why Do We Need Aggregate Functions?

The principal never asks "show me every row." The principal asks:

- ▶ **How many** students do we have?
- ▶ What is the **average** age?
- ▶ What is the **total** fee collected?

An **aggregate function** takes many rows and returns **one single value**.

```
SELECT COUNT(*) FROM students;           -- 5
SELECT AVG(age) FROM students;           -- 21.00
SELECT MIN(age), MAX(age) FROM students; -- 19, 23
SELECT SUM(age) FROM students;           -- 105
```

Function	Answers the question
COUNT(*)	how many rows?
SUM(col)	what is the total?
AVG(col)	what is the average?
MIN(col) / MAX(col)	smallest / biggest value?

Aggregates: Details Beginners Miss

```
-- COUNT(*) counts rows, COUNT(col) skips NULLs
SELECT COUNT(*) FROM students;           -- 6 (with Anita)
SELECT COUNT(grade) FROM students;       -- 5 (Anita's NULL
    skipped)

-- aggregates + WHERE: filter first, then calculate
SELECT AVG(age) FROM students
WHERE city = 'Kathmandu';                -- 19.5

-- rename the result with AS
SELECT COUNT(*) AS total_students FROM students;
```

- ▶ WHERE runs **first** (keeps Kathmandu rows), the aggregate runs on what remains.
- ▶ NULL is ignored by SUM, AVG, MIN, MAX, and COUNT(col). Only COUNT(*) sees every row.

GROUP BY: One Result Per Group

Why? "Count students **per city**" needs one number **for each** city, not one number total.

```
SELECT city, COUNT(*) AS total
FROM students
GROUP BY city;
```

How GROUP BY works, step by step:

1. make one bucket per distinct city
2. drop each row into its bucket
3. run COUNT(*) inside each bucket
4. output one row per bucket

city	total
Kathmandu	2
Pokhara	2
Chitwan	1

Rule: every column in SELECT must be either inside an aggregate or listed in GROUP BY.

HAVING and the General Order

WHERE filters rows **BEFORE** grouping. **HAVING** filters groups **AFTER** grouping.

```
-- cities having more than 1 student
SELECT city, COUNT(*) AS total
FROM students
GROUP BY city
HAVING COUNT(*) > 1;           -- Kathmandu 2, Pokhara 2

-- combined: joins + aggregates
SELECT s.name, COUNT(c.course_id) AS courses
FROM students AS s
LEFT JOIN courses AS c ON s.student_id = c.student_id
GROUP BY s.student_id, s.name
ORDER BY courses DESC;
```

General order

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ...
ORDER BY ... ;
```

Why not WHERE for groups? WHERE runs before groups exist, so it cannot see COUNT(*).

Practice Set 4: Aggregates (Q1–Q5)

Use your `shop.products` and `library` tables from before.

Report questions, first half

1. Total number of products, cheapest price, and most expensive price in one query.
2. Total stock of all products, and total stock of Electronics only.
3. Average price per category (GROUP BY).
4. Count products per category, but show only categories with at least 2 products. WHERE or HAVING? Why?
5. Average price per category, counting only products cheaper than 60000. Which filter goes in WHERE?

Practice Set 4: Aggregates (Q6–Q10)

Report questions, second half

6. Categories where the average price is above 1000, sorted biggest first. Use the full general order.
7. Library: total fine per member (join + GROUP BY). Members with no loans must show 0. Which join?
8. Library: members whose total fine is above 100 (HAVING).
9. COUNT(*) vs COUNT(fine): make one fine NULL and show the difference.
10. In one sentence: why can HAVING see COUNT(*) but WHERE cannot?

Remember: SELECT ... FROM ... WHERE ... GROUP BY ...
HAVING ... ORDER BY ...;

Recap: The Complete Learning Path



- ▶ Companies need SQL because business data must be safe, searchable and rule-protected.
- ▶ CRUD = INSERT, SELECT, UPDATE, DELETE. Never forget WHERE in UPDATE and DELETE.
- ▶ PK is the identity of a row (solves the two Ram problem). FK connects tables and blocks orphan records.
- ▶ Joins: inner keeps matches, left/right keep one full side, full keeps everything. Always ask whose rows must all appear.
- ▶ Aggregates + GROUP BY + HAVING turn rows into reports.
- ▶ You built four databases yourself: shop, hospital, library, and reports on top.

Thank you!